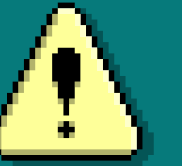
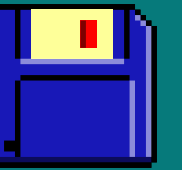
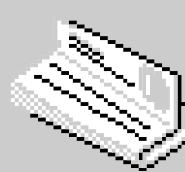
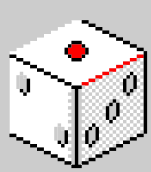
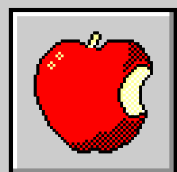


FPGA VGA Graphics System



Digital Video Interface,
VGA Timing Controller
& Hardware Graphics
Engine



11:11PM

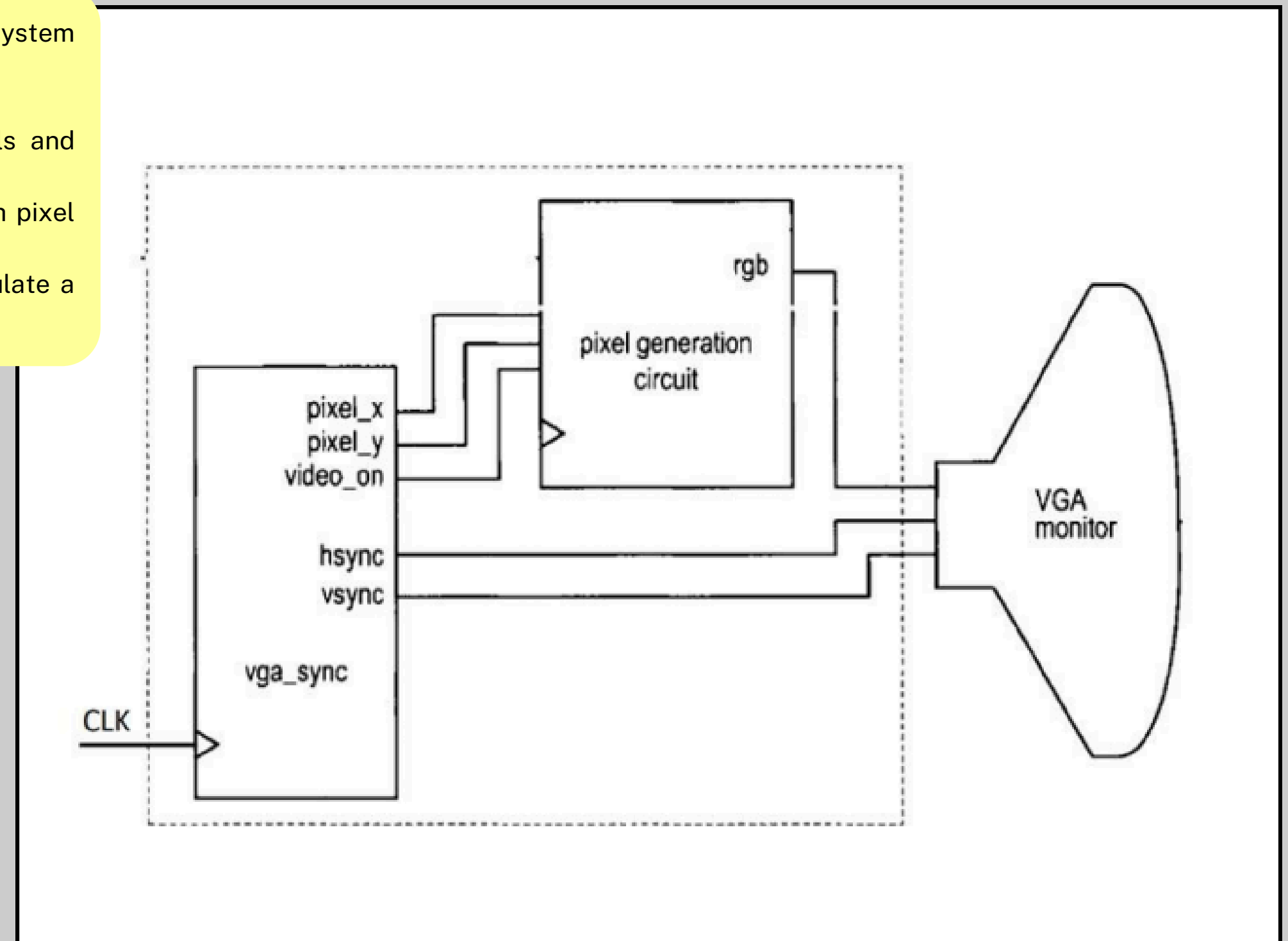
The objective: is to design and implement a complete VGA-based graphics system on the Cyclone IV FPGA platform.

The project is divided conceptually into two main layers:

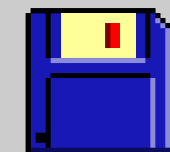
1. **VGA Controller** — responsible for generating the VGA timing signals and identifying the current pixel position.

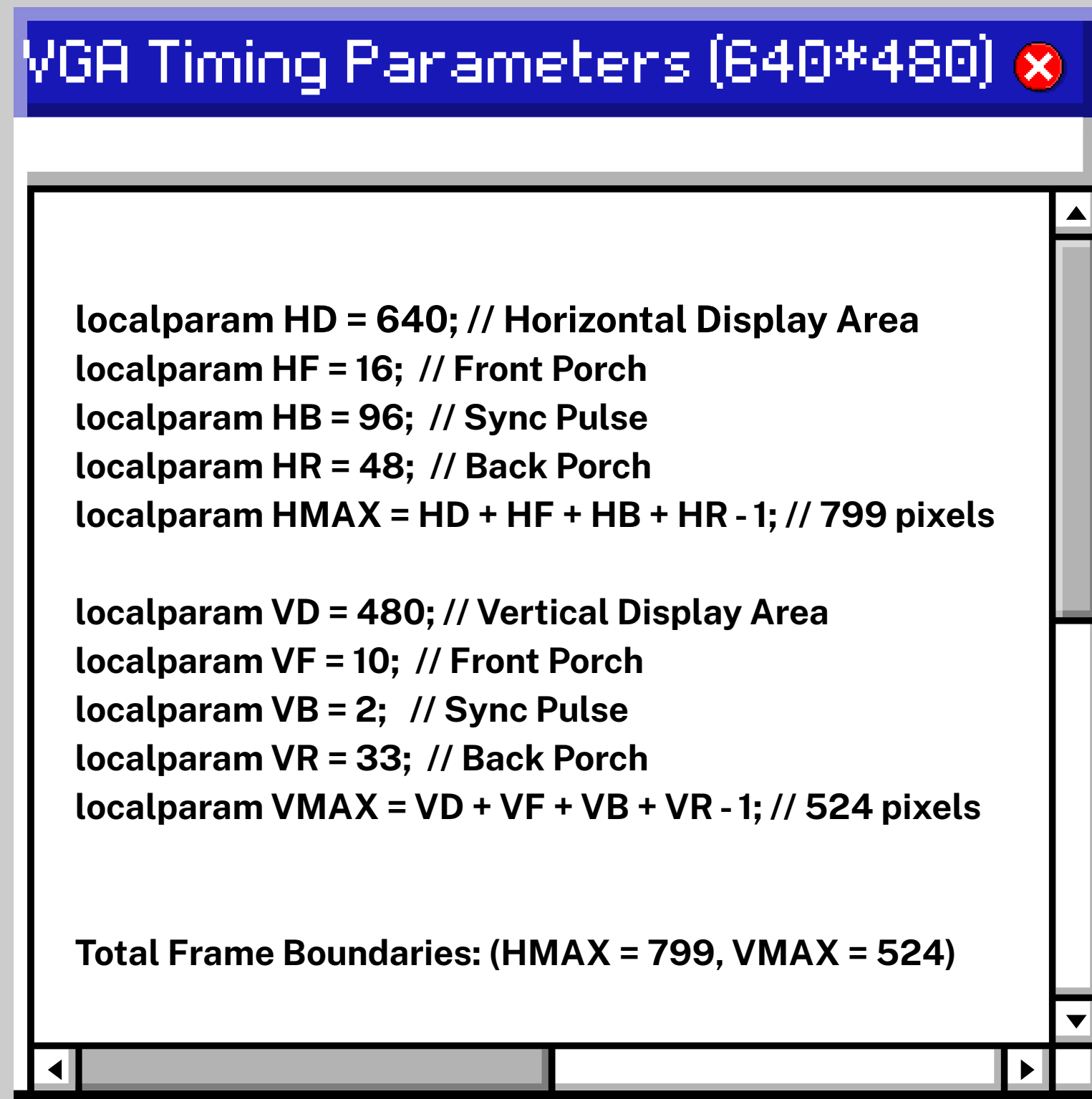
2. **Graphics Engine** — responsible for determining the RGB value of each pixel based on the current pixel coordinates.

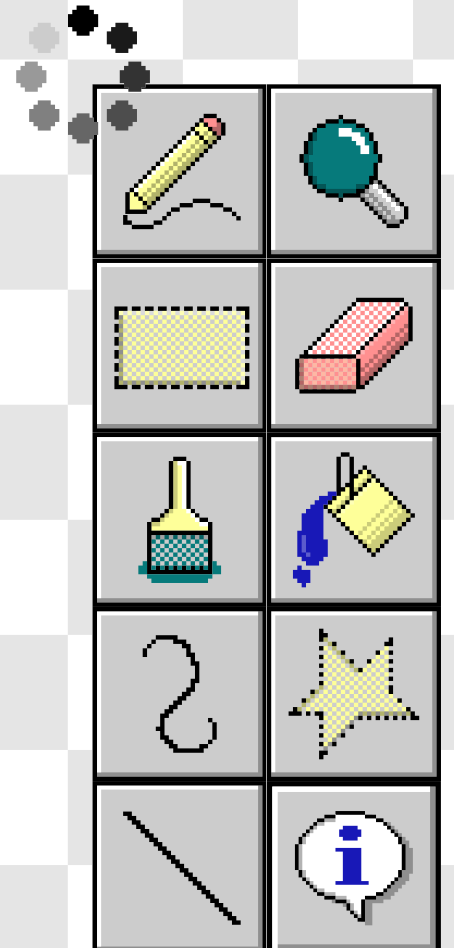
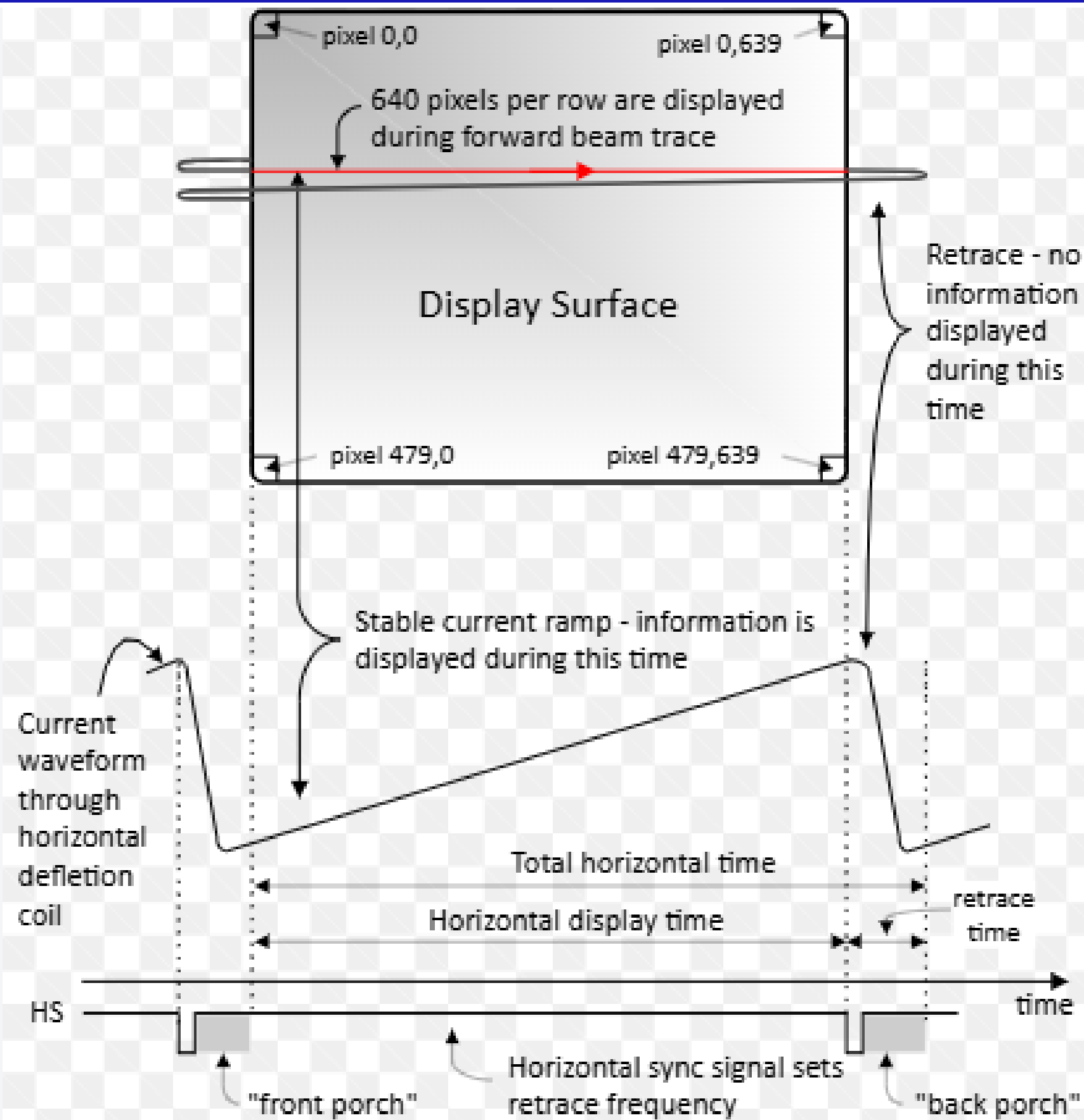
The goal is to understand how digital hardware can generate and manipulate a video stream in real time.



Required System Architecture







Clock Divider

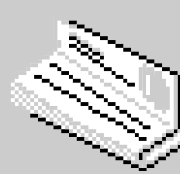
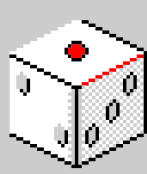
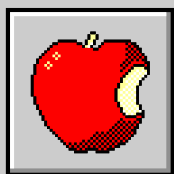
```
REG PIXEL_TICK;  
ALWAYS @(POSEDGE CLK OR POSEDGE RESET) BEGIN  
  IF (RESET)  
    PIXEL_TICK <= 1'B0;  
  ELSE  
    PIXEL_TICK <= ~PIXEL_TICK; // 50MHZ -> 25MHZ  
END
```

Pixel Counters

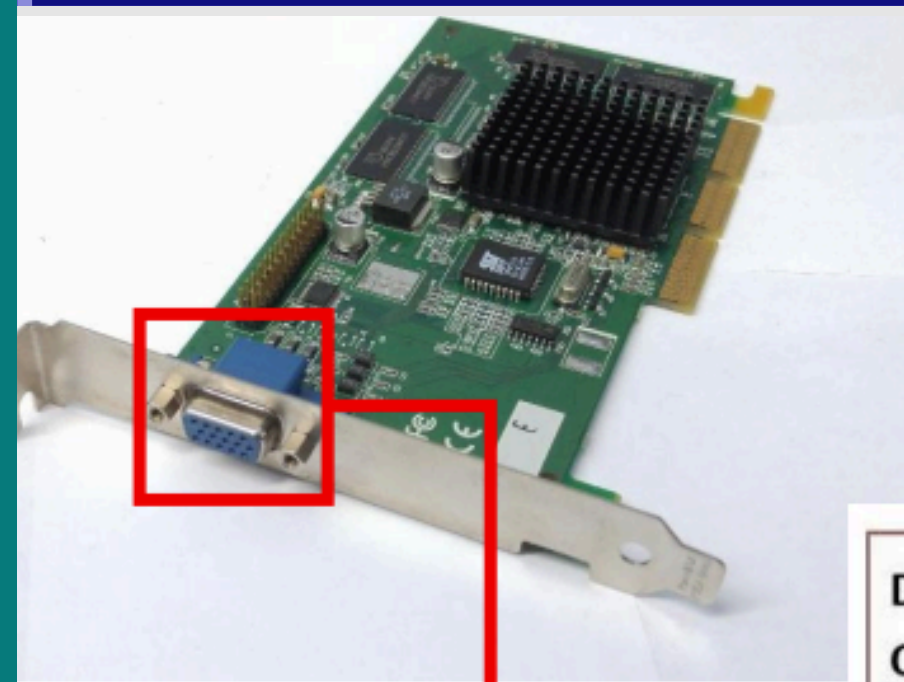
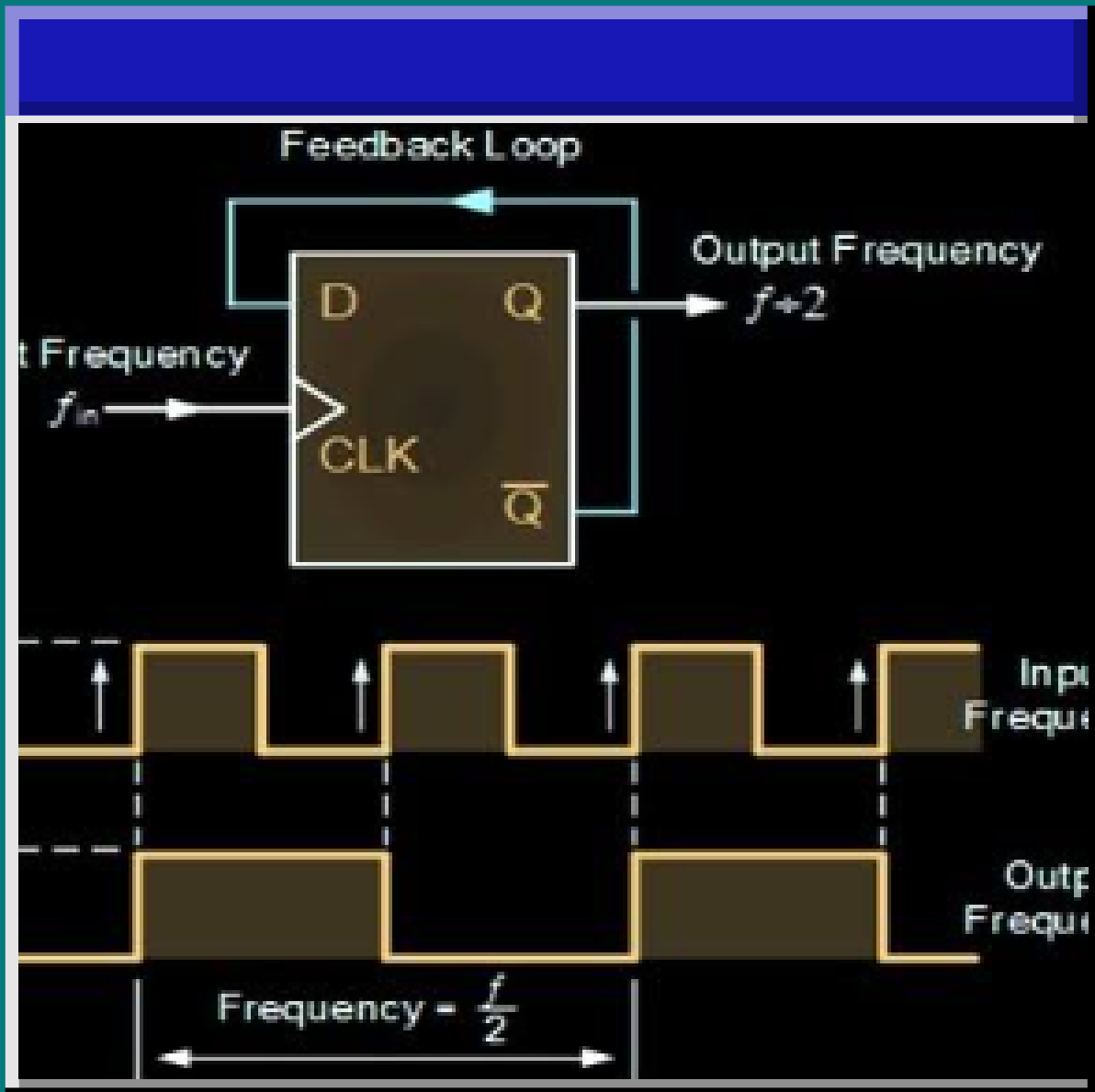
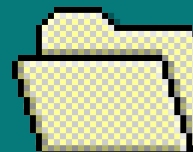
```
reg [9:0] h_count_reg, h_count_next;  
reg [9:0] v_count_reg, v_count_next;  
always @(posedge clk or posedge reset)  
begin  
  if (reset) begin  
    h_count_reg <= 0;  
    v_count_reg <= 0;  
  end else if (pixel_tick) begin  
    h_count_reg <= h_count_next;  
    v_count_reg <= v_count_next;  
  end  
end
```

Next-state logic

```
always @(*) begin  
  h_count_next = h_count_reg;  
  v_count_next = v_count_reg;  
  
  if (h_count_reg == HMAX) begin  
    h_count_next = 0;  
    if (v_count_reg == VMAX)  
      v_count_next = 0;  
    else  
      v_count_next = v_count_reg + 1'b1;  
  end else begin  
    h_count_next = h_count_reg + 1'b1;  
  end  
end
```



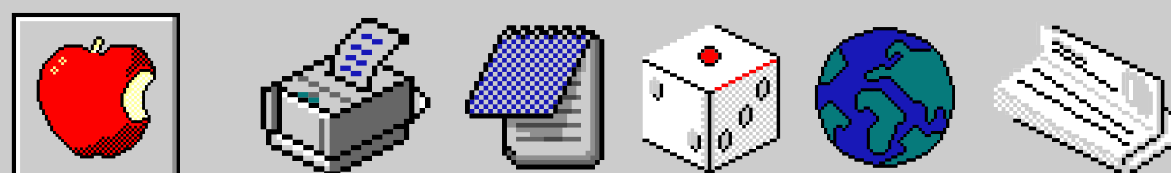
[Back to Agenda Page](#)



DIGITAL COMMUNITY

standard DE-15 VGA connector and its RGB/HSYNC/VSING pins.

DB-15 Connector	VGA Signals	VGA port, view from Wire Side
	Vertical Sync(VS)	
	Horizontal Sync(HS)	
	Blue	
	Green	



[Back to Agenda Page](#)

Sync Signals & Outputs



// Sync Signal Generation (Active-Low)

```
always @(posedge clk or posedge reset) begin
```

```
...//generate h,v-sync as active-low signals
```

```
hsync <= ~((h_count_reg >= (HD + HF)) &&  
(h_count_reg < (HD + HF + HB)));
```

```
vsync <= ~((v_count_reg >= (VD + VF)) &&  
(v_count_reg < (VD + VF + VB)));
```

```
end
```

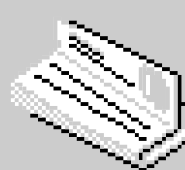
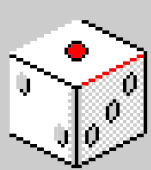
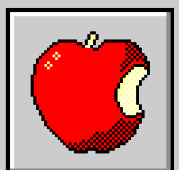
// Video On Signal (high only inside visible screen)

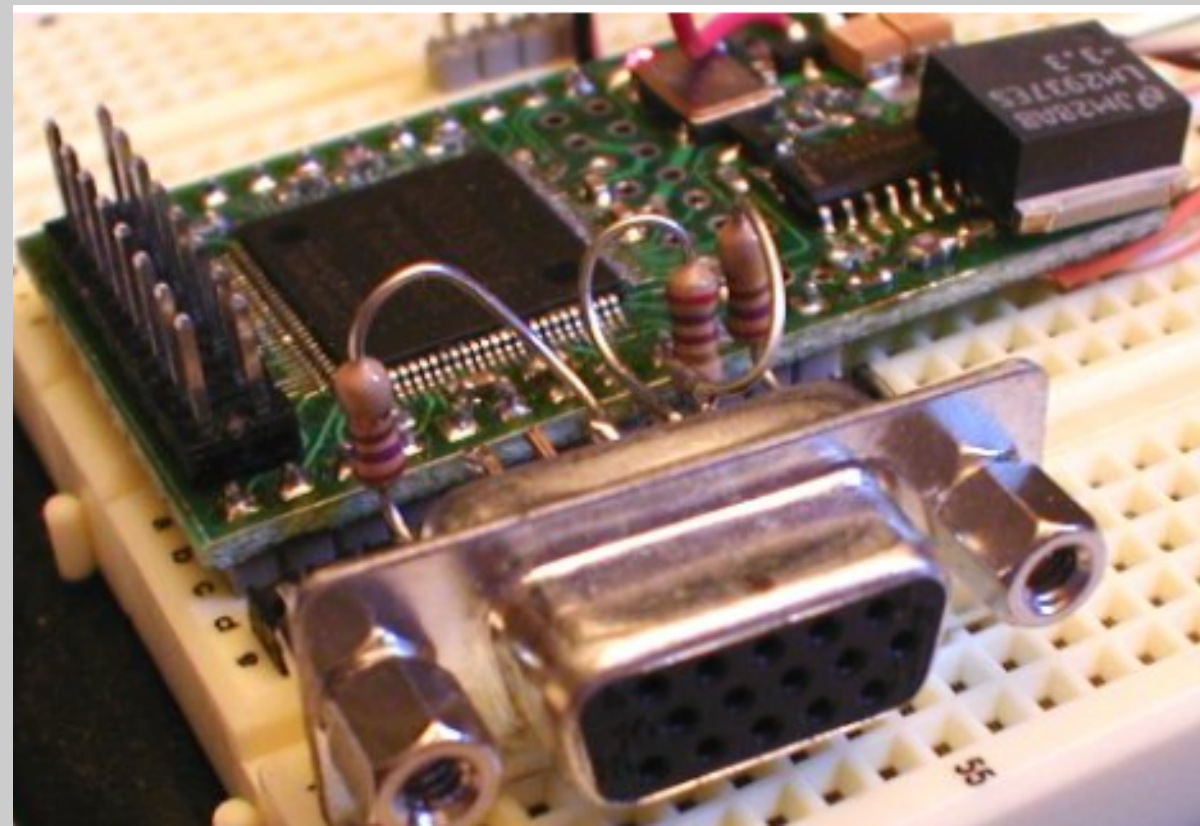
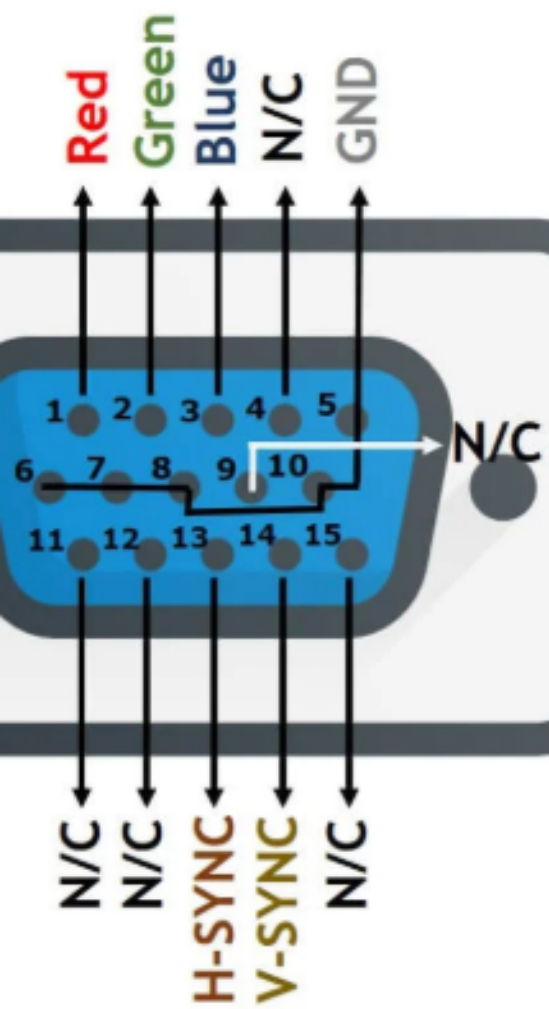
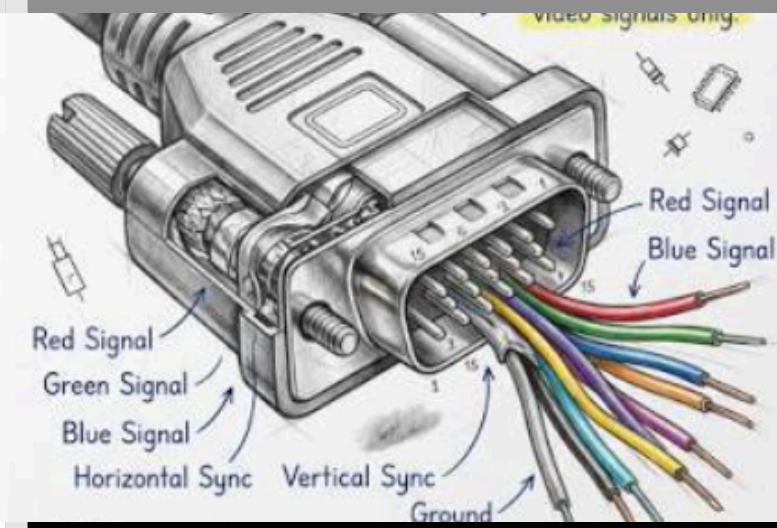
```
assign video_on = (h_count_reg < HD) && (v_count_reg  
< VD);
```

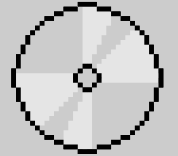
// Output Pixel Coordinates

```
assign pixel_x = h_count_reg;
```

```
assign pixel_y = v_count_reg;
```







Graphics Engine :



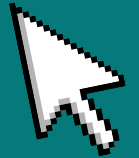
responsible for determining the RGB value of each pixel based on the current pixel coordinates.

[Back to Agenda Page](#)

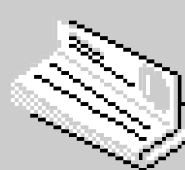
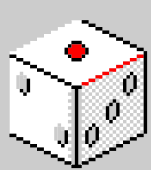
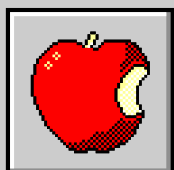
✕ 🔍

```
module pixel_generation (  
    input clk,  
    input [9:0] pixel_x,  
    input [9:0] pixel_y,  
    input video_on,  
    output reg [2:0] rgb  
);  
  
    reg video_on_delayed;  
    wire [14:0] rom_address;  
    wire [2:0] rom_color;  
  
    wire [9:0] img_x;  
    wire [9:0] img_y;  
  
    ROM R (  
        .address(rom_address),  
        .clock(clk),  
        .q(rom_color)  
    );  
  
    assign img_x = pixel_x >> 2;  
    assign img_y = pixel_y >> 2;  
  
    assign rom_address = (img_y * 15'd160) + img_x;
```

pixel_generation.v



```
        always @(posedge clk) begin  
            video_on_delayed <= video_on;  
  
            if(!video_on_delayed) begin  
                rgb <= 3'b000;  
            end else begin  
  
                rgb <= rom_color;  
  
            end  
        end  
    endmodule
```



[Back to Agenda Page](#)

3-Bit RGB Color Mapping

With just one bit per color channel (Red, Green, Blue), we unlock a fundamental palette of 8 distinct colors. This minimalist approach demonstrates direct hardware control over visual output at the most basic level.

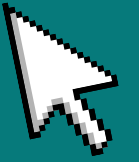


[Back to Agenda Page](#)

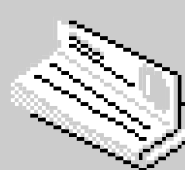
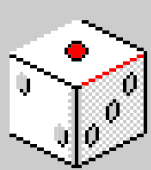
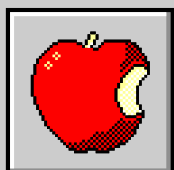
✕ 🔍

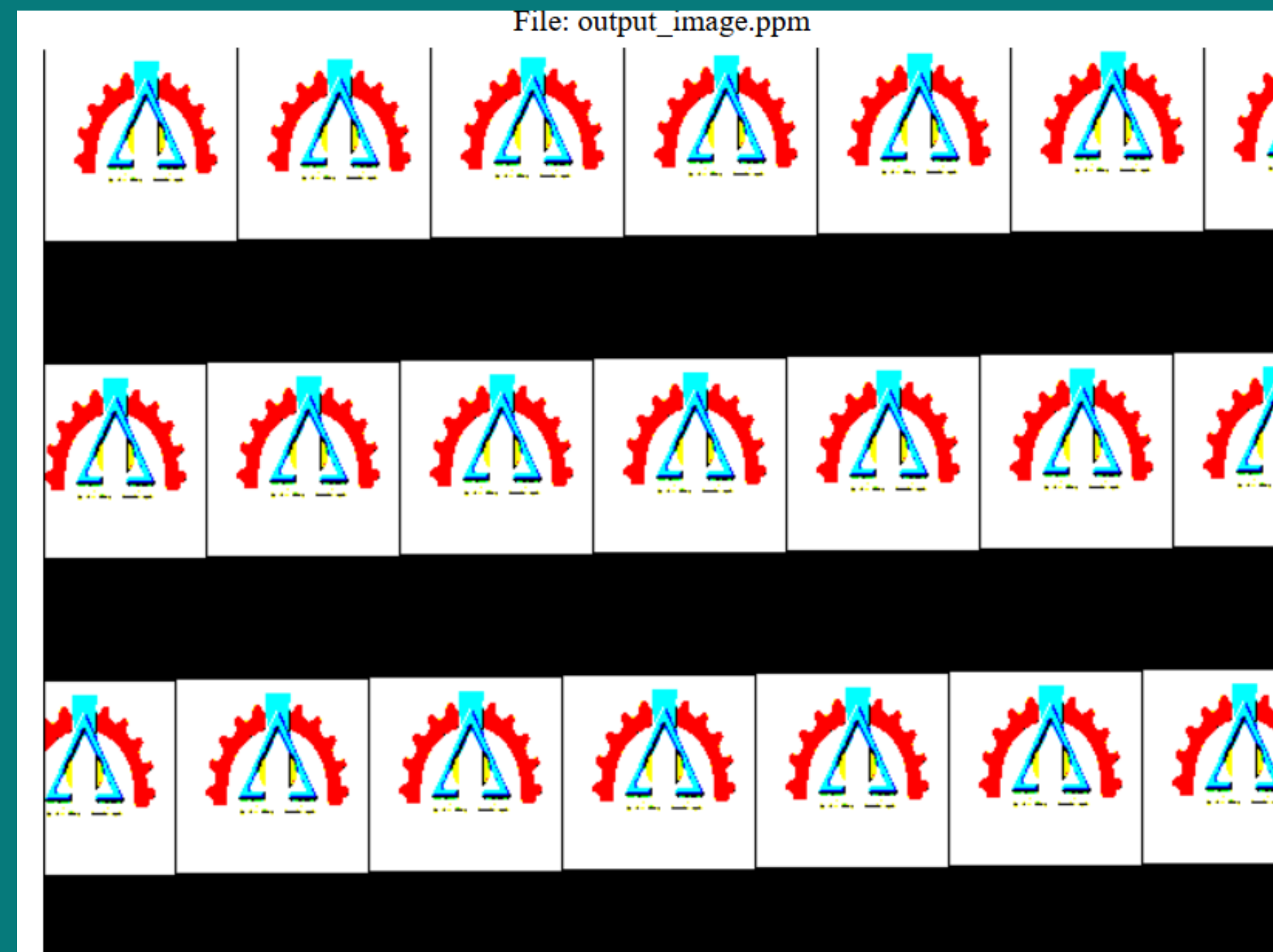
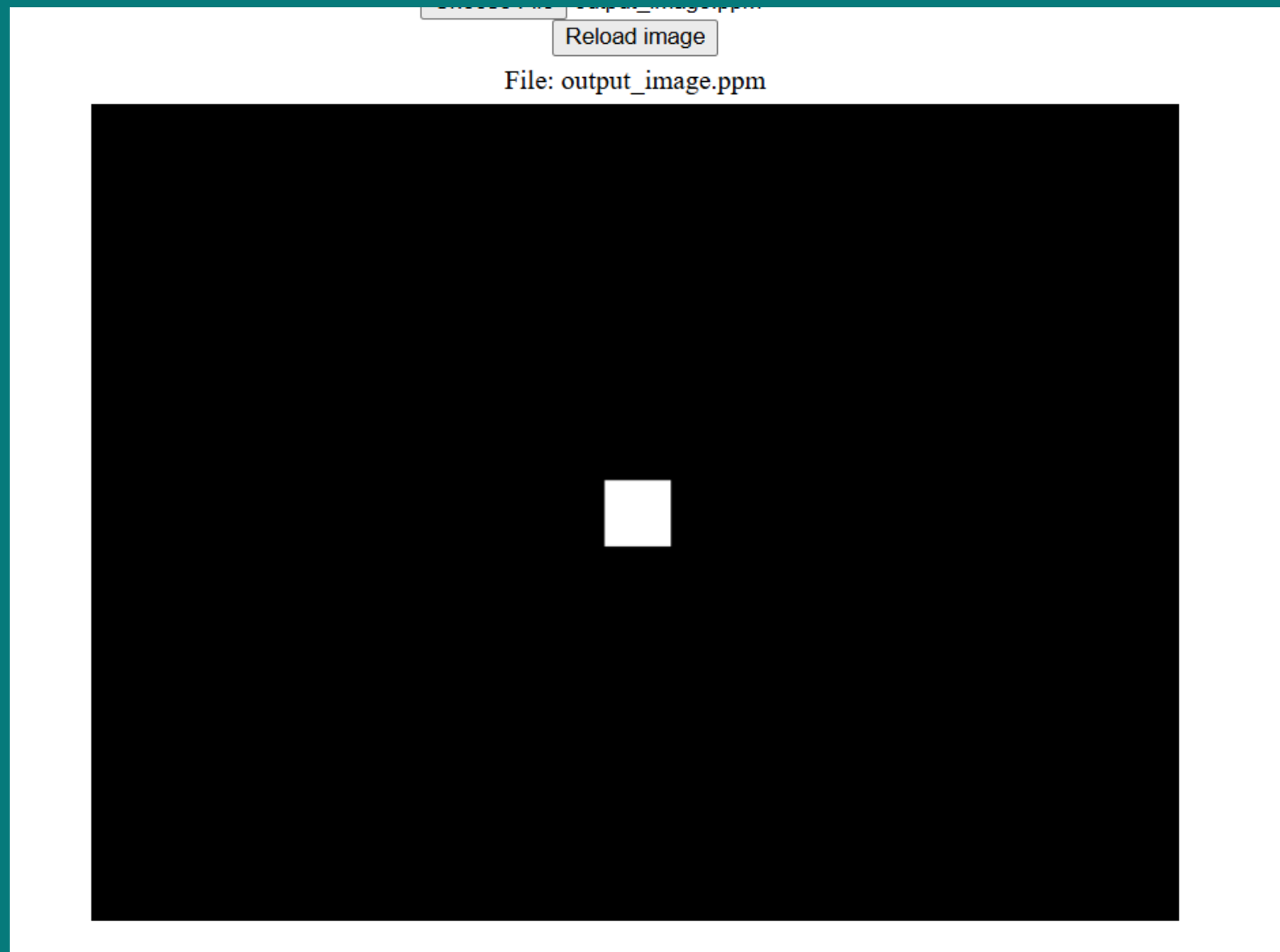
```
module VGA(  
    input wire clk,  
    input wire reset,  
    output reg  hsync,  
    output reg  vsync,  
    output wire [2:0] rgb  
);  
  
    wire video_on;  
    wire [9:0] pixel_x;  
    wire [9:0] pixel_y;  
    wire hsync_internal;  
    wire vsync_internal;  
  
    vga_sync sync_unit (  
        .clk(clk),  
        .reset(reset),  
        .hsync(hsync_internal),  
        .vsync(vsync_internal),  
        .video_on(video_on),  
        .pixel_x(pixel_x),  
        .pixel_y(pixel_y)  
    );  
endmodule
```

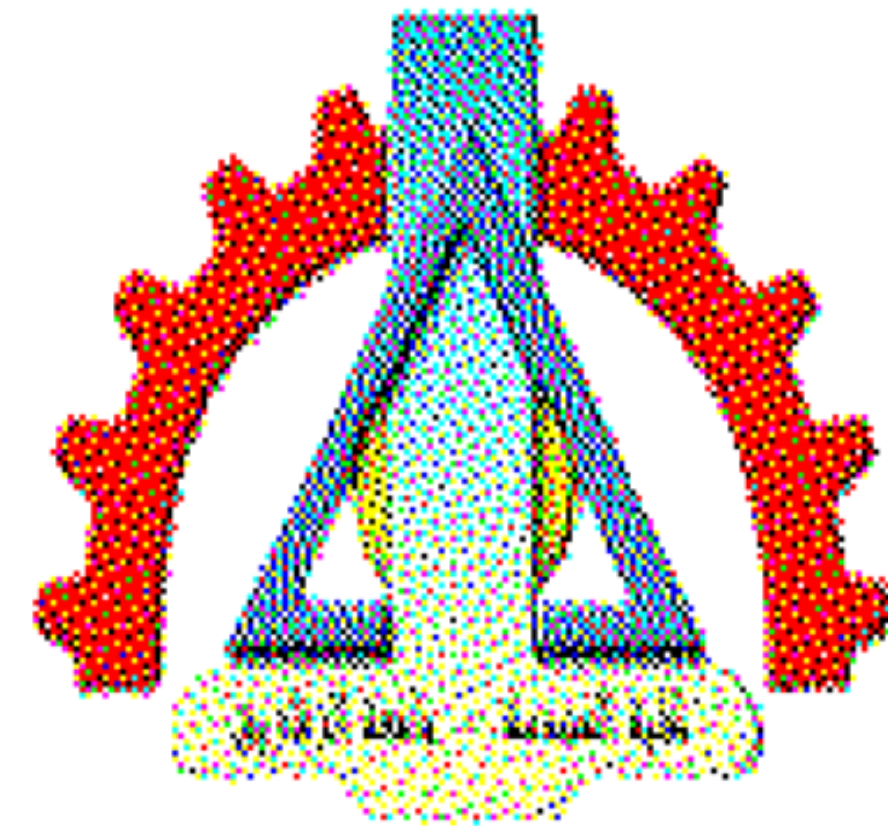
VGA.v



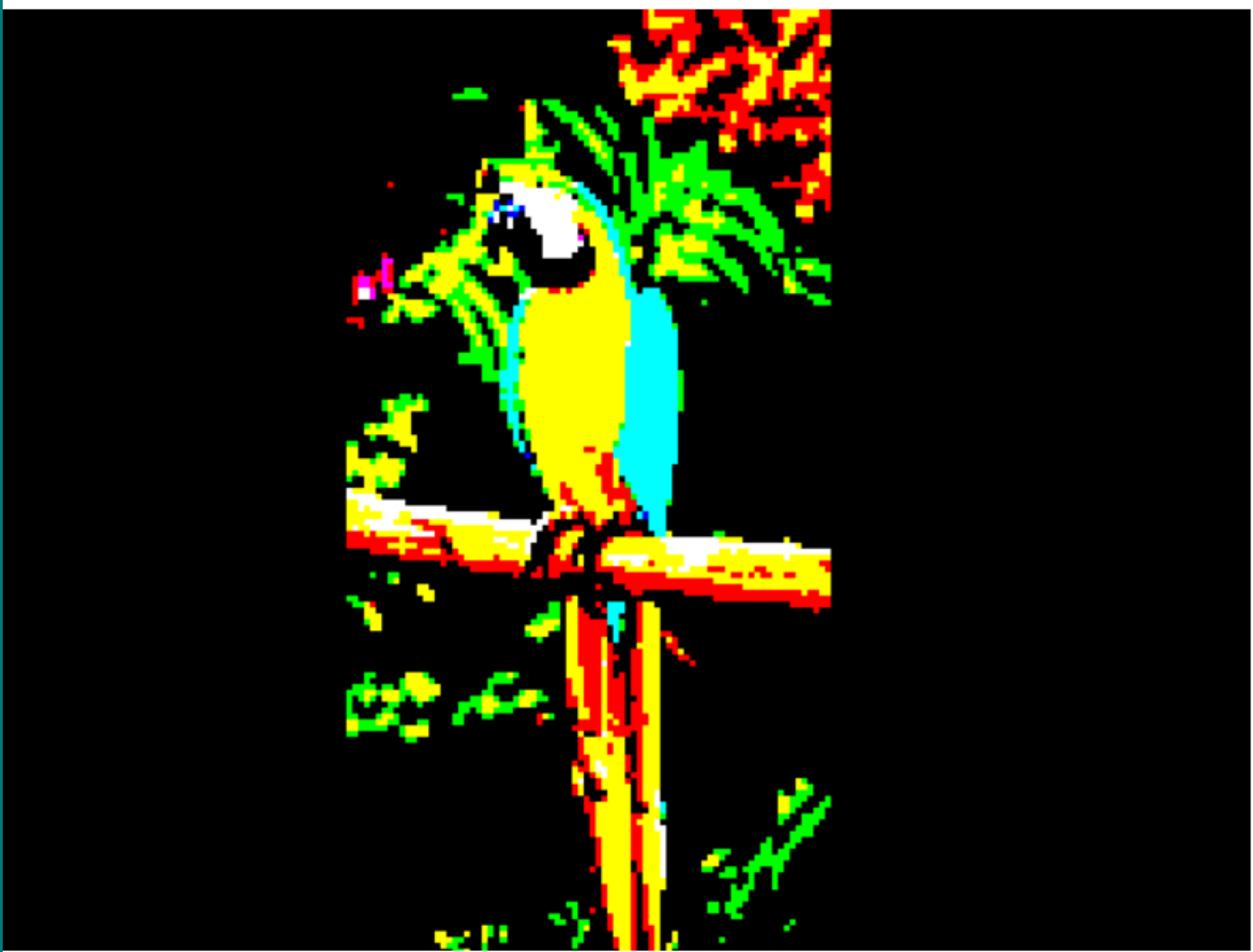
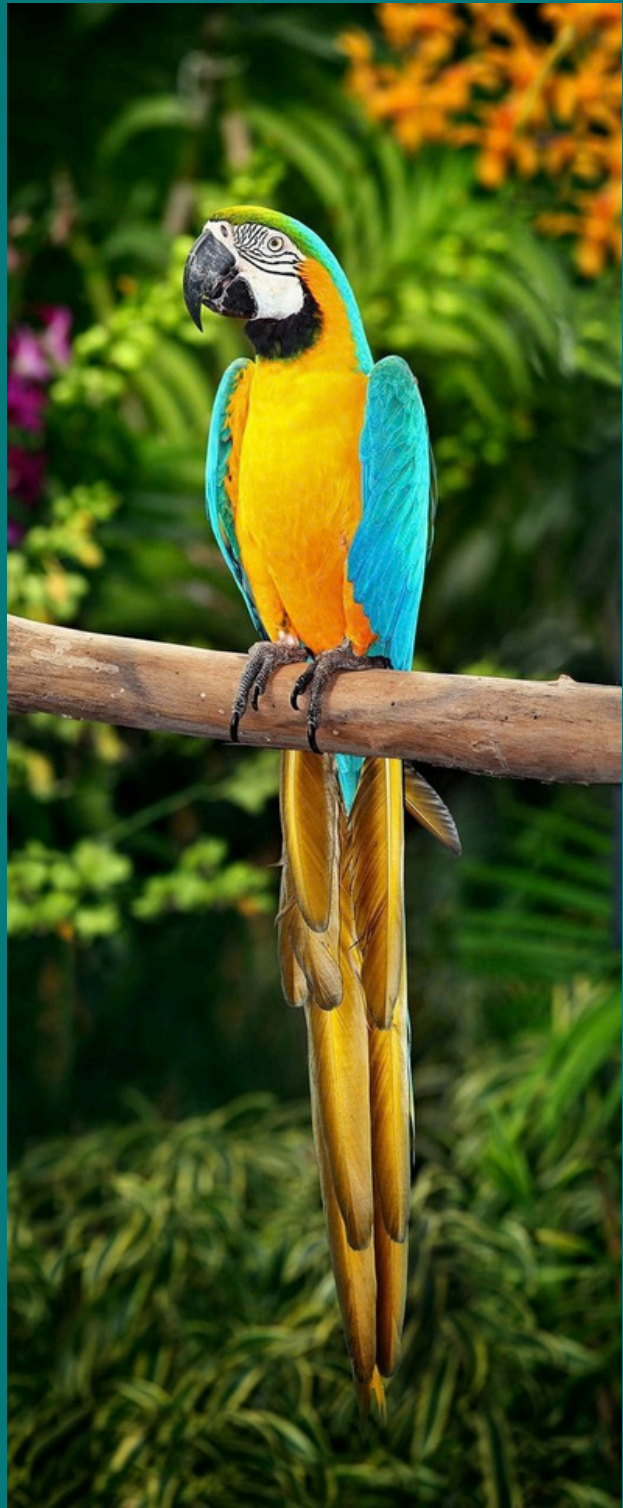
```
    pixel_generation pixel_unit (  
        .clk(clk),  
        .video_on(video_on),  
        .pixel_x(pixel_x),  
        .pixel_y(pixel_y),  
        .rgb(rgb)  
    );  
  
    always @(posedge clk) begin  
        hsync <= hsync_internal;  
        vsync <= vsync_internal;  
    end  
endmodule
```











File: output_image.ppm





